Module 3

UART — Protocol + RTL + Basic Testbench

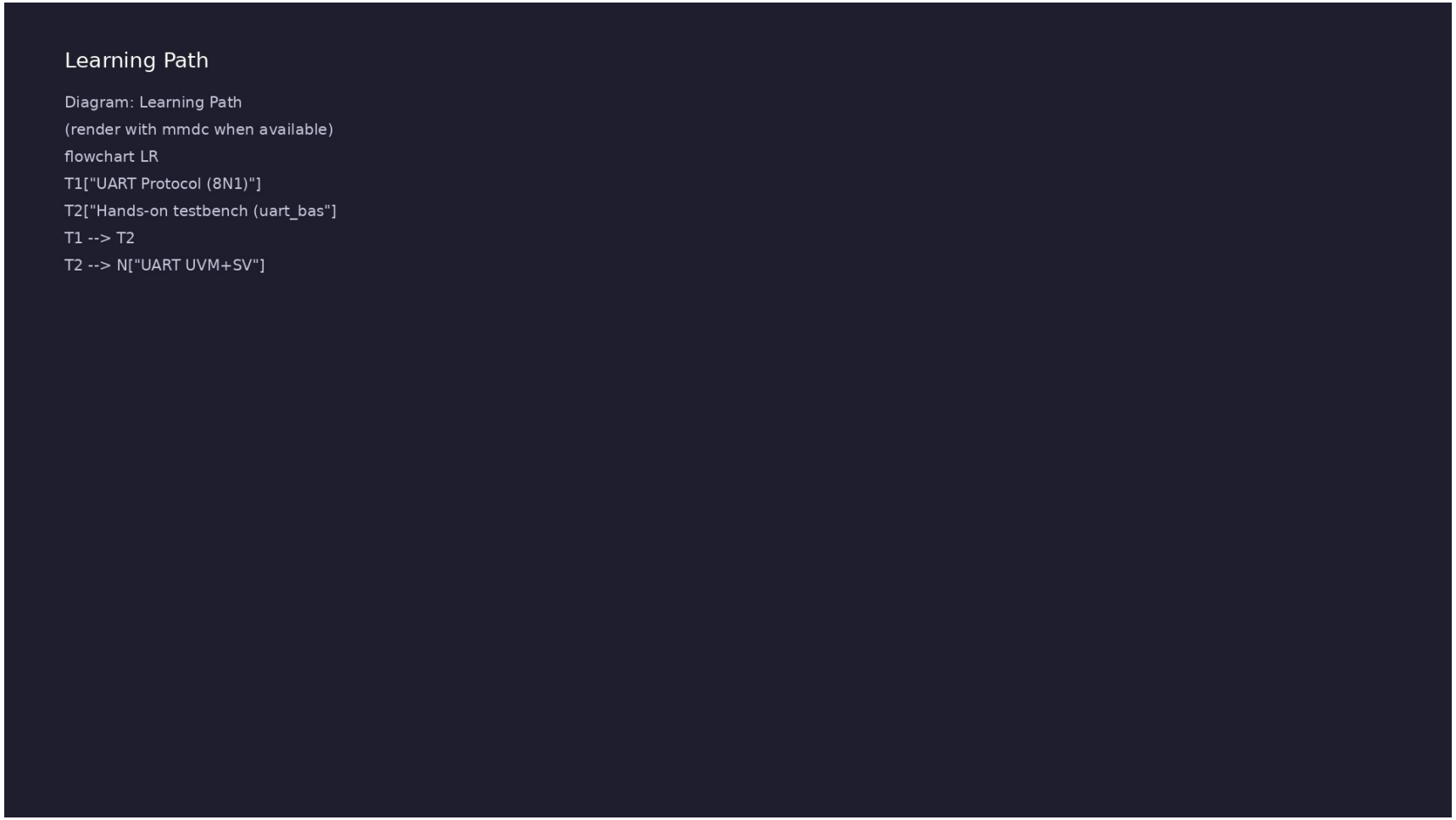
Learning objectives

- UART 8N1: Start bit (0), 8 data bits (LSB first), stop bit (1); idle high.
- Baud rate: Role of a baud generator (clock divider → baud_tick).
- RTL architecture: TX (parallel→serial, shift per baud_tick), RX (start detect, sample, data_valid),…
- Basic TB: Loopback, directed stimulus, and checking without UVM.

Prerequisites

- Module 1 (spec → RTL) and Module 2 (basic TB → UVM, toolchain).
- Verilator, Make, C++ compiler. No UVM required for the uart_baseline example.

Learning path



Understand the UART protocol (8N1, baud), translate it to RTL (TX, RX, baud gen), and verify with a **basic** (non-UVM) directed testbench.

Overview

- Module 3 is the first **protocol** module:

How to learn this module

Suggested learning path

- Read the detailed UART learning guide:

  UART_LEARNING_GUIDE.md — what UART is, what kind of protocol…
- Read the protocols + UVM overview:

  LEARNING_GUIDE_PROTOCOLS_AND_UVM.md — Part B § 4. When to Use Ba…

Design architecture

1. Block hierarchy

- top_uart_baseline → `baud_gen`, `uart_tx`, `uart_rx`; sim_main.cpp drives `clk`/`rst_n` only.
- Loopback: `assign rx = tx` — one wire connects transmitter to receiver.

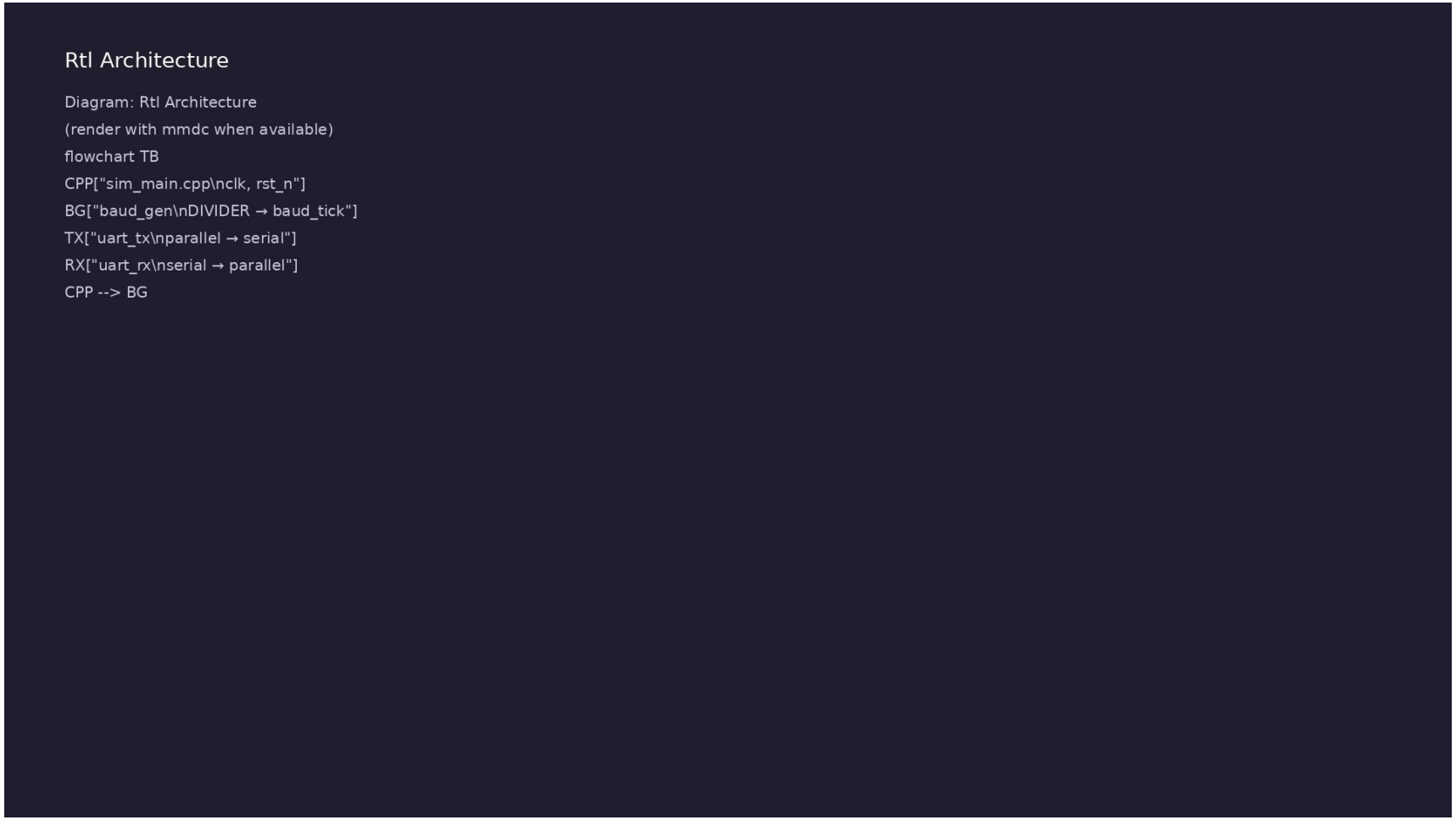
2. RTL blocks

- baud_gen: `DIVIDER` → `baud_tick` every N clocks; shared by TX and RX.
- uart_tx: parallel-in / serial-out; start + 8 data (LSB first) + stop; `busy` during frame.
- uart_rx: start detect, sample per `baud_tick`, `data_valid` + optional `framing_error`.

3. Timing and clocking

- Single `clk` domain; async `rst_n`; all bit times referenced to `baud_tick`.

RTL block diagram (reference)



Module 3: DUT hierarchy and signal flow.

Verification / testbench diagram (reference)

Module 3: stimulus, observation, and checking.

uart_tx — 8N1 frame FSM

```
// Simple UART transmitter (8N1, no parity) for teaching.
// This version uses an external baud_tick enable (from baud_gen)
// so that each bit lasts many clk cycles in real time, while the
// internal state machine still advances once per baud_tick.

`timescale 1ns/1ps

module uart_tx (
    input  wire       clk,
    input  wire       rst_n,      // Active-low async reset
    input  wire       baud_tick,  // One-cycle pulse at baud rate
    input  wire       start,      // Pulse high for one cycle to start
transmit
    input  wire [7:0] data_in,    // Byte to transmit, LSB first on the
line
    output reg        tx,         // UART TX line (idle high)
    output reg        busy        // High while frame is in progress
```

uart_rx — start detect & sample

```
// Simple UART receiver (8N1, no parity) for teaching.
// This version uses an external baud_tick enable (from baud_gen)
// so that each bit lasts many clk cycles in real time, while the
// RX state machine advances once per baud_tick.
//
// Protocol:
//   - Idle line is high.
//   - Start bit is a single low bit.
//   - 8 data bits follow, LSB first, one per clock.
//   - Stop bit is a single high bit.
//
// This RX block watches the serial line for a start bit, then samples
// one bit per clock to reconstruct the byte. When a full frame has
// been received, it pulses `data_valid` for one cycle with `data_out`.

`timescale 1ns/1ps
# ...
```

baud_gen — divider → baud_tick

```
// Simple baud-rate generator for teaching.
// Divides the input clock down to a 1-cycle-wide baud_tick pulse.
//
// Example: For a 50 MHz system clock and 115200 baud target,
// DIVIDER ≈ 50_000_000 / 115200 ≈ 434.

`timescale 1ns/1ps

module baud_gen #(
    parameter int DIVIDER = 434  // Default divider, can be overridden
) (
    input  wire clk,
    input  wire rst_n,      // Active-low async reset
    output reg  baud_tick   // One-cycle pulse at baud rate
);
    int count;
# ...
```

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator compiles RTL + SystemVerilog testbench into a C++ model
- sim_main.cpp: generates clk/rst_n, calls eval() until $finish
- Directed test (initial block or C++): drive stimulus, wait for DUT flags
- Self-check: compare outputs; print PASS/FAIL (see terminal demo slide)
- Repo path: module3/examples/<example>/ — run make run from that directory

Example directory layout

- dut/: uart_tx.v, uart_rx.v, baud_gen.v
- top_uart_baseline.sv: Loopback (rx=tx), baud_gen, uart_tx, uart_rx; directed test in initial block
- sim_main.cpp: Clock, reset; run until $finish

Directed test execution sequence (1)

- wait(rst_n)
- pulse start/data_in
- wait(!busy)
- wait(data_valid)
- compare data_out

Directed test execution sequence (2)

- repeat
- $finish.

Makefile — uart_baseline run

```
# Makefile for Module 3 UART baseline (basic testbench, no UVM)
#
# Usage:
#   make run
#

VERILATOR ?= verilator
TOPLEVEL ?= top_uart_baseline
DUT_SRC = dut/*.v
RTL_SRC = top_uart_baseline.sv $(DUT_SRC)
HARNESS = sim_main.cpp

VERILATOR_FLAGS = -sv --timing -Wall -Wno-fatal -Wno-TIMESCALEMOD -Wno-
WIDTHTRUNC

.PHONY: run compile clean

#
```

Key files to study

Open these in the repo

- `module3/examples/uart_baseline/dut/baud_gen.v` — divider → `baud_tick`
- `module3/examples/uart_baseline/dut/uart_tx.v` — 8N1 transmit
- `module3/examples/uart_baseline/dut/uart_rx.v` — receive and `data_valid`
- `module3/examples/uart_baseline/top_uart_baseline.sv` — loopback + directed test
- `docs/UART_LEARNING_GUIDE.md` — protocol reference

Verification & testing methods

1. Verification goals

- Prove RTL implements UART 8N1 before UVM (Module 4).
- Happy-path loopback: bytes sent on TX appear correctly on RX.

2. Baseline directed test

- Stimulus: `initial` block — `start`, `data_in` (0x55, 0xAA).
- Check: `wait(data_valid)`; compare `data_out`; `$error` / `$display`.
- No UVM: pin wiggling only — fastest way to learn the protocol on real RTL.

3. Coverage gaps (defer to Module 4)

- Random baud, framing errors,
  `busy` back-to-back, functional
  coverage.
- Map checks to [UART_LEARNING_GUI
  DE.md]
(UART_LEARNING_GUIDE.md)
  spec bullets.

Loopback directed test

```
// Basic directed test: wait for reset release, send bytes, check RX
    initial begin
        start   = 1'b0;
        data_in = 8'h00;

        wait (rst_n === 1'b1);
        repeat (10) @(posedge clk);

        // Send 0x55
        @(posedge clk);
        start   = 1'b1;
        data_in = 8'h55;
        @(posedge clk);
        start   = 1'b0;
        wait (busy === 1'b0);
        wait (data_valid === 1'b1);
# ...
```

Syllabus topics

Protocol & design details

Detail: UART quick reference (8N1) (1)

- Framing: `idle=1` → `start=0` → 8 data bits (LSB first) → `stop=1`.
- Baud math: `DIVIDER = round(clk_hz / baud)`. Example: 50 MHz clock, 115200 baud → divider ≈ 434.
- Sampling: Basic design samples once per bit at the `baud_tick`. Production UARTs often oversample (…
- Reset state: Line idles high, TX not busy, RX clears internal shift register, `data_valid` low.

Detail: UART quick reference (8N1) (2)

- Errors: `framing_error` when stop bit is not high at expected sample; optional `parity_error` if pa…

Detail: TX design checklist

- Inputs: `clk`, `rst_n`, `baud_tick`, `start`, `data_in[7:0]`.
- Outputs: `tx`, `busy`.
- Behavior: On `start`, latch `data_in`, drive start bit low, then shift out bits [0:7] each `baud_ti...
- Edge cases: Ignore `start` while `busy` (or queue it in a FIFO if you extend the design).

Detail: RX design checklist

- Inputs: `clk`, `rst_n`, `baud_tick`, `rx`.
- Outputs: `data_out[7:0]`, `data_valid` (pulse), `framing_error`.
- Behavior: Detect falling edge for start; wait one `baud_tick` to sample mid-start; sample 8 bits on…
- Robustness options: Add metastability sync on `rx`; add oversampling + majority vote; add `rx_ready…

Detail: Baud generator notes

- Inputs: `clk`, `rst_n`, `divisor` (constant parameter in the baseline).
- Output: `baud_tick` high for one cycle every `divisor` clocks.
- For synthesis, keep the counter simple (`divisor-1` down-counter or up-counter compare). For sim, y…

Detail: Protocol extensions (not in baseline example) (1)

- Data bits: 5–9 data bits are common.
- Parity: Even/odd/mark/space; adds a parity bit before stop.
- Stop bits: 1 or 2 stop bits.
- Flow control: RTS/CTS hardware pins (outside the serial line) to throttle traffic.

Detail: Protocol extensions (not in baseline example) (2)

- The baseline RTL assumes 8N1 with no parity and no flow control.

Verification flow (reference)

Stimulus, DUT response, and check points for this module.

1. UART Protocol (8N1) (1/4)

- Signals: Serial line (tx/rx), idle high; system clock; baud_tick (one pulse per bit time).
- Frame: 1 start bit (0), 8 data bits (LSB first), 1 stop bit (1).
- Timing: One bit per baud_tick; baud_tick derived from system clock via a divider (e.g. DIVIDER = cl…
- Framing: `idle=1` → `start=0` → 8 data bits (LSB first) → `stop=1`.
- Baud math: `DIVIDER = round(clk_hz / baud)`. Example: 50 MHz clock, 115200 baud → divider ≈ 434.
- Sampling: Basic design samples once per bit at the `baud_tick`. Production UARTs often oversample (…

1. UART Protocol (8N1) (2/4)

- Reset state: Line idles high, TX not busy, RX clears internal shift register, `data_valid` low.
- Errors: `framing_error` when stop bit is not high at expected sample; optional `parity_error` if pa…
- Inputs: `clk`, `rst_n`, `baud_tick`, `start`, `data_in[7:0]`.
- Outputs: `tx`, `busy`.
- Behavior: On `start`, latch `data_in`, drive start bit low, then shift out bits [0:7] each `baud_ti…
- Edge cases: Ignore `start` while `busy` (or queue it in a FIFO if you extend the design).

1. UART Protocol (8N1) (3/4)

- Inputs: `clk`, `rst_n`, `baud_tick`, `rx`.
- Outputs: `data_out[7:0]`, `data_valid` (pulse), `framing_error`.
- Behavior: Detect falling edge for start; wait one `baud_tick` to sample mid-start; sample 8 bits on…
- Robustness options: Add metastability sync on `rx`; add oversampling + majority vote; add `rx_ready…
- Inputs: `clk`, `rst_n`, `divisor` (constant parameter in the baseline).
- Output: `baud_tick` high for one cycle every `divisor` clocks.

1. UART Protocol (8N1) (4/4)

- For synthesis, keep the counter simple (`divisor-1` down-counter or up-counter compare). For sim, y…
- Data bits: 5–9 data bits are common.
- Parity: Even/odd/mark/space; adds a parity bit before stop.
- Stop bits: 1 or 2 stop bits.
- Flow control: RTS/CTS hardware pins (outside the serial line) to throttle traffic.
- The baseline RTL assumes 8N1 with no parity and no flow control.

2. Hands-on testbench (uart_baseline)

- Loopback: Connect TX output to RX input; send bytes from TX, check they appear on RX.
- Directed test: Reset release, send 0x55 and 0xAA, wait for `data_valid`, check `data_out`. Implemen…
- Self-check flow: `wait(rst_n)` → pulse `start`/`data_in` → `wait(!busy)` → `wait(data_valid)` → com…

Hands-on examples

Module 3 self-check

```
$ ./scripts/module3.sh --help
```

Example 1: UART baseline loopback

- UART 8N1 TX/RX RTL with shared `baud_gen`
- Loopback wiring (`rx = tx`) and directed bytes 0x55 / 0xAA
- Baseline self-check without UVM (wait for `data_valid`, compare `data_out`)

Demo: UART baseline loopback

```
$ cd module3/examples/uart_baseline && make run
```

Key concepts

1. Asynchronous serial

- No dedicated clock wire; TX and RX must share the same bit time (`baud_tick`).
- Loopback (`rx = tx`) proves TX serialization and RX deserialization together.

2. 8N1 frame

- Idle high → start (0) → 8 data bits LSB first → stop (1).
- `busy` on TX; `data_valid` pulse on RX when a byte is ready.

Common pitfalls

Mismatched baud timing

- Mistake: Different dividers on TX and RX in a loopback test.
- Correct: One `baud_gen` drives both (as in uart_baseline).
- Why: RX samples at the wrong bit times.

Only checking `busy`

- Mistake: Assuming success when TX finishes.
- Correct: Wait for `data_valid` and compare `data_out` to the sent byte.
- Why: `busy` does not prove the loopback byte is correct.

Practice & assessment

What you should know

- Describe UART 8N1 frame and baud timing.
- Explain the role of uart_tx, uart_rx, and baud_gen.
- Run the uart_baseline example and interpret the loopback test.
- Ready for Module 4: UART UVM+SV verification.

Exercises

- Run uart_baseline
- Trace one byte
- Baud and divider
- Optional: Waveforms

Assessment checklist

- Can describe UART 8N1 (start bit, 8 data LSB first, stop bit) and the role of baud_tick.
- Can explain the role of uart_tx, uart_rx, and baud_gen in the RTL.
- Can run uart_baseline and interpret the loopback test result.
- Ready for Module 4: UART UVM+SV verification.

Summary & next steps

- Understand the UART protocol (8N1, baud), translate it to RTL (TX, RX, baud gen), and verify with a…
- Complete module3/CHECKLIST.md
- Review module3/EXAMPLES.md and run each lab
- Next: UART UVM+SV